RAG Pipeline — Build Summary

What we built, how the pieces fit together, what went wrong along the way, and what we tested to confirm it works.

1. The Stack, Ground Up

- Runs on a small **AWS EC2 server** (3.8GB RAM, 2 vCPU, 28GB disk) — a genuinely tight box, a recurring constraint throughout this project.
- **Docker Compose** runs three containers: Postgres (database), LiteLLM (proxy routing AI model calls to OpenAI), and Hermes (the agent framework).
- Hermes runs multiple isolated "profiles" — separate AI personalities with their own instructions and tool access. **researcher** is the profile we equipped with the knowledge-base pipeline described below.
- **Postgres + pgvector** (a search extension) is where all ingested knowledge actually lives, as searchable numeric representations of text called embeddings.

2. The Pipeline We Built

One script, **rag_ingest.py**, handles getting information into the knowledge base. It supports four sources:

- **wiki** — fetch and store a Wikipedia article.
- **file** — ingest an uploaded PDF, Word doc, PowerPoint, text file, or image (photo/scan).
- **db** — pull free-text rows from an existing database table (e.g. a support-ticket log).
- **dbrow** — pull rows from a database table where the useful information is spread across several columns (e.g. a product or equipment table).

Every mode ends the same way: break the text into pieces, embed each piece, and store it. A second script, **rag_query.py**, does the reverse — embeds a question and finds the closest-matching stored pieces, citing where each came from. The researcher agent decides, on its own, when to check this knowledge base versus the live web or general training knowledge — that logic lives in SOUL.md.

3. Content-Aware Chunking

Early on, every piece of content was split the same way — flat paragraph chunks, regardless of what the document actually looked like. Now, before splitting, each file, page, or slide is first classified by its structural shape rather than its file format — plain prose, a table of facts, a Q&A list, numbered legal sections, a dialogue, step-by-step instructions, a header-sectioned document, or an email — and only then split using whichever method fits that shape (FAQ per question-answer pair, legal per numbered section, transcripts keeping speaker turns grouped), instead of one-size-fits-all treatment.

For text that doesn't clearly match any of those eight shapes, the system asks the AI to suggest a one-off splitting pattern, sanity-checks the result, retries once if it looks wrong, and falls back to the original flat splitter as a last resort so ingestion never fails outright — still occasionally inconsistent right at the "ordinary prose" vs "not real text" boundary, noted in Section 8.

Planned upgrade: once server hardware allows (see Section 9), the plan is still to replace the format-specific extractors (pypdf, python-docx, python-pptx, Tesseract) with a single library, **unstructured**, for layout-aware extraction. That's a separate, still-pending change from the chunking-strategy work described above.

4. What Hermes Contributes

Standard RAG is a fixed round-trip: fetch text that looks relevant, paste it into the prompt, generate an answer. No judgment involved — it always retrieves, always answers, and can't tell a strong match from a weak one.

- **Hermes decides for itself** whether to check the knowledge base at all, rather than doing it automatically.
- **It retries when results look weak**, rather than settling for the first mediocre match.
- **It chooses between tools** — knowledge base, live web search, or direct file read — instead of one fixed pipeline.
- **One pipeline, many roles.** The same retrieval system wires into different Hermes profiles at once, without duplicating the retrieval machinery.
- **It answers like an analyst.** Its instructions require citing sources and flagging thin evidence, rather than confidently stitching together a plausible response.

Without Hermes, none of the above happens on its own — a plain LLM call is a single, stateless round-trip with no terminal, no file access, and no ability to notice a weak match or choose between tools.

5. What Each Piece Is Responsible For

- **pypdf / python-docx / python-pptx** — extract raw text from PDF, Word, and PowerPoint files.
- **Tesseract (OCR)** — reads text out of photos and scanned pages with no selectable text layer.
- **psycopg2** — lets the pipeline pull rows from any database table as a knowledge source.
- **Postgres / pgvector** — stores and searches the embeddings.
- **LiteLLM** — the single gateway every embedding, chat, and content-classification request passes through.

6. Where We Went Wrong — and How We Fixed It

- Tried swapping to a more "battle-tested" parsing library (unstructured). Its PDF support silently pulled in a multi-gigabyte machine-learning stack and filled the server's disk, breaking the pipeline. **Fix:** reverted to the original lightweight libraries — already the right tools for this job.
- A text-splitting rule built for prose silently deleted short-but-important lines (prices, single-line OCR text) by assuming anything under 40 characters was noise. **Fix:** added a second, filter-free splitting method for structured or short-form content.
- A routine server restart wiped every manually installed tool (OCR engine, parsing libraries) with no warning, since they lived outside the one folder that survives a restart. **Fix:** a boot script now reinstalls everything automatically on every start; confirmed with a real restart test.
- The AI assistant sometimes skipped its own knowledge base and answered from general knowledge instead, in one case inventing details not present in the data. **Fix:** rewrote its instructions to require a knowledge-base check first for any plausibly relevant question.
- Building the new content-classification step (Section 3) hit two silent failures: a classification request used a setting the AI model doesn't support, but an overly broad error handler swallowed the real error and made every test look like a generic failure; separately, a missing code import would have crashed several new splitting methods the moment they were used, despite testing fine in isolation. **Fix:** added a raw diagnostic to see the real error, then re-tested against the live pipeline, not just isolated pieces.

- The query script (rag_query.py) was silently cutting every retrieved result down to its first 300 characters before showing it to Hermes, no matter how long the actual stored chunk was. This surfaced during real-world testing (Section 7): a full sprint-retro transcript was cut off mid-sentence, and Hermes never saw the second half of it at all. **Fix:** removed the cutoff so full chunks are returned; confirmed with a direct re-query and a repeat of the failed question, both now correct.

7. Tests We Ran

- **Format coverage:** ingested a real Wikipedia article, a real 9-page PDF, synthetic Word/PowerPoint files, and multiple database tables — confirmed correct retrieval for each.
- **OCR:** generated a photo of a promotional flyer, confirmed the pipeline correctly read and retrieved its text.
- **Disaster recovery:** deliberately forced a full server-container rebuild to confirm the persistence fix holds up under a real wipe, not just in theory.

Real-world Q&A test: loaded a themed test set (a fictional gym's equipment guide PDF, orientation handbook, staff training deck, two equipment placard photos, and a maintenance database table, all describing the same business) and asked the researcher agent 13 questions, from simple lookups to ones requiring several sources at once. Most answers were accurate and well-cited: placard text, staff procedures, legal terms, and maintenance facts across five database rows were all reported correctly and sourced from the right document. It also handled "not in the data" cases well, honestly reporting no membership-freeze or parental-leave policy existed rather than guessing.

Two problems turned up. First, two of five maintenance answers cited an extra, incorrect source alongside the correct one — the underlying facts were right, but the source attribution was sloppy; this did not reproduce on a retest, so it may have been run-to-run variance rather than a fixable bug. Second, and more seriously, a full sprint-retro transcript was sitting in the knowledge base as a single, complete chunk — including the exact detail the question asked about — but the agent reported the file as unreadable and gave an incomplete answer. Investigating traced this to a real bug in rag_query.py, which was silently cutting every result down to 300 characters before Hermes ever saw it. Fixed (Section 6) and confirmed working: re-asking the same question after the fix returned a complete, correctly-cited answer.

8. Still Open

- The prose-vs-unclassifiable boundary in the content-aware chunking step (Section 3) is occasionally inconsistent — worth tightening the classification prompt further.
- OCR for scanned PDF pages is built but not yet tested against a real scanned document.
- The cheap pre-check added in Section 10 is regex-based — testing it against a wider real-world sample set (Section 11) already caught and fixed two more collisions, so this is an area that will likely keep needing attention as more content types get thrown at it.

9. A Note on "unstructured" (the library we moved away from)

unstructured is genuinely the best available option for parsing PDF, Word, PNG, and JPG files — real layout understanding, table detection, and built-in OCR, tested at scale in the open-source community. Not usable here purely due to hardware: its PDF/image support pulls in a machine-learning stack needing more disk and memory than this box has. Worth revisiting if upgraded.

10. Retrieval Quality and Cost Upgrades

After reviewing a separate project's codebase for ideas, three improvements were added to the query and classification side of the pipeline.

- **Reranking:** the query script used to return a fixed top 5 closest-matching chunks by similarity alone. It now pulls a wider pool of 15 candidates, then a second, cheap AI pass judges which are actually relevant to the specific question before narrowing back down to 5. This directly fixed the equipment-log bug from Section 7/8 — re-running that same test afterward returned all five equipment rows correctly, where one had been getting crowded out before.
- **Cheap pre-check before classification:** obviously-shaped content (FAQ, legal, procedural, transcript, email) is now detected with a fast, free pattern check before ever calling the AI classifier, which is only used when the shape genuinely isn't obvious (prose, tabular data, header-sectioned documents, or anything ambiguous).
- **Prompt caching:** the fixed classification instructions are now sent as a separate, unchanging system message instead of being rebuilt into a new prompt on every call, so the AI provider can cache and reuse that fixed portion.

The pre-check introduced one bug during testing: its dialogue-detection pattern (a capitalized word followed by a colon) also matched email headers like "From:" and "To:", briefly misclassifying a test email as a transcript. **Fix:** email detection now runs first, and the dialogue pattern excludes common header words. Re-tested against nine varied samples afterward, all correct.

11. Switching to a Cheaper Model for Classification and Reranking

The classifier and reranker (Section 10) were both running on gpt-5-mini. Since both jobs are fairly mechanical — labeling a document's shape, or judging whether a passage answers a question — we compared pricing against smaller, cheaper models built for exactly this kind of task, and against a dedicated reranking-only service (Cohere). The dedicated service turned out to cost more at our scale, since it charges a flat fee per search rather than by token, and our documents are small. Switching both jobs to gpt-5-nano, a smaller model from the same provider, was the clear win — noticeably cheaper, and no new integration required.

The switch surfaced two more issues, both found and fixed the same day through testing rather than left for later:

- **Reranking silently returned nothing at first.** The smaller model uses more hidden "reasoning" tokens than the previous one when judging a large batch of candidates, and was running out of budget before writing an actual answer. **Fix:** raised the token budget for that one call; confirmed working afterward.
- **Two more pre-check collisions.** Retesting with a fuller set of samples showed FAQ text ("Q:" / "A:" lines) and plain key-value data ("Price:", "Category:", etc.) were both being mistaken for dialogue, the same class of bug fixed in Section 10 for email headers. **Fix:** tightened the same detection logic further.

Retested against the real gym maintenance data afterward: reranking now correctly excludes equipment with "no issues reported," which the previous model had been including — a small accuracy improvement alongside the cost savings.

12. Hybrid Search and Real PDF Tables

Hybrid search: the query script previously only searched by meaning (embeddings). It now also runs a keyword search (Postgres full-text search) in parallel and merges both sets of results before reranking, so

exact terms — names, IDs, exact phrases — that embeddings sometimes fuzzy-match are also caught directly. Tested with both an exact equipment name and a natural-language question; both returned the correct result.

Real PDF tables: the PDF reader was swapped from pypdf to pdfplumber, which can detect table boundaries the same way our Word-document handling already did. Tables are now extracted separately, in clean rows and columns, instead of getting flattened into the surrounding paragraph text — closing the PDF-table gap noted in Section 8. Tested against a synthetic pricing PDF containing both a table and regular paragraphs on the same page: the table extracted correctly, and a follow-up question pulled and cited the exact row.

Found while testing: Hermes skipped its own knowledge base on a generically-worded question, judging from the wording alone that it probably wasn't covered — a resurfacing of the knowledge-base-skipping issue from Section 6. **Fix:** tightened SOUL.md to require an actual search attempt before deciding a topic is out of scope, instead of guessing from phrasing. Confirmed fixed by re-asking the same question.

Found, not fixed: a further retest with an apostrophe in the question ("what's") returned zero results, even though the identical question worked when run directly. Traced to how Hermes's own terminal tool builds its shell command — a single quote in the question text breaks the quoting. This lives inside the packaged Hermes agent itself, not in our scripts, so it's noted here as a known quirk rather than something we could patch directly.

13. Contextual Retrieval and Automated Regression Testing

Contextual retrieval: a short AI-written note is now generated for each chunk before it's embedded, summarizing what document and section that chunk came from (e.g. "this table lists membership pricing tiers from the gym's pricing page"). This is a published technique (Anthropic's own engineering write-up reports large gains in retrieval accuracy from it) aimed at a specific weakness: a short, isolated chunk -- a bare table row, a one-line fact -- carries very little meaning on its own once separated from its surroundings. The note is generated using the same document as a fixed prompt prefix across all of that document's chunks, so the AI provider can reuse (cache) that portion instead of resending it every time. Applied across every ingestion mode, including database rows. Tested against both a document (a PDF with a table) and a database row -- both produced accurate, genuinely useful context notes, confirmed working through a live Hermes query afterward.

Automated regression testing: until now, confirming the pipeline still worked after a change meant manually asking Hermes real questions and checking the answers by hand. That's now been turned into a saved set of question/expected-result pairs (regression_tests.json) and a runner script (run_regression_tests.py) that checks our retrieval code directly -- not Hermes's final wording, just whether the right source material comes back. Seeded from the real, previously human-verified gym test questions, plus two covering the newer features (hybrid search and reranking's exclusion of healthy equipment). All 6 checks pass as of this update; this file should be re-run after any future pipeline change to catch regressions automatically instead of re-testing everything by hand.

14. Groundedness / Retrieval Quality Flag

Previously, Hermes had to judge whether a result set was good enough by eyeballing raw numeric distance values -- an implicit, easy-to-miss signal. The reranker (Section 10) now also grades the whole result set in the same AI call it already makes, labeling it STRONG, WEAK, or NONE, and that label is printed as an explicit flag right after the question. SOUL.md was updated so Hermes treats WEAK or NONE as insufficient

evidence and falls back to a live search instead of forcing an answer, rather than relying on its own read of distance numbers. This follows the same idea as Corrective RAG (CRAG), a published pattern for grading retrieved content before generating an answer -- adapted here as a lightweight addition to a call we were already making, rather than a separate grading step.

Tested with a well-covered question (correctly flagged STRONG) and one with no real answer in the knowledge base (correctly flagged NONE, no results forced). Regression suite re-run afterward: one test failed on the first pass and passed cleanly on a second run with no code changes in between -- since the reranker is itself an AI call, some run-to-run flakiness in this specific test is expected and not a hidden bug; noted here rather than glossed over.

15. Semantic Caching

A new table, rag_query_cache, stores recent questions (as embeddings) alongside their retrieval quality and results. Before running the full pipeline, an incoming question is checked against this cache -- if a near-duplicate question was already answered recently, the cached result is returned immediately, skipping the vector search, keyword search, and the rerank/quality AI call entirely (the most expensive steps). This helps when the same or similarly-worded questions get asked repeatedly, which is common for a small business's day-to-day questions.

The similarity threshold was calibrated empirically rather than guessed: two different phrasings of the same question ("how much is the premium plan" vs. "what is the cost of the premium plan") measured about 0.18 cosine distance apart, while a genuinely different question measured about 0.83 -- so a 0.25 threshold catches real paraphrases with comfortable margin against false matches. Cached entries expire after 60 minutes so a cached answer can't quietly go stale if new data gets ingested in the meantime, and entries older than 24 hours are cleaned up automatically so the cache table doesn't grow forever. Tested with a direct repeat, a paraphrase (confirmed cache hit, correct result returned instantly), and a genuinely different question (confirmed it ran fresh, no false match). Regression suite re-run afterward, 6/6 passing.